

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

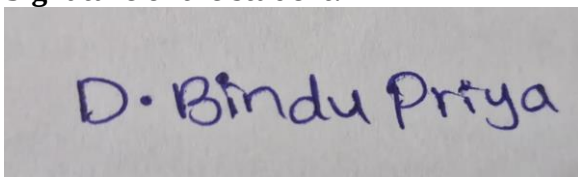
Marks: 50

Code of Conduct:

I D.BINDU PRIYA(192425162) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTIONS:10

Total Marks:50

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

6. Perceptron and Multilayer Perceptron (MLP)

- a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.
- b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.
- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

- a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.
- b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Experiment 1: Learning Algorithms (Linear Regression using Gradient Descent)

Aim

To develop a Linear Regression model using Gradient Descent and study how the error reduces as training progresses.

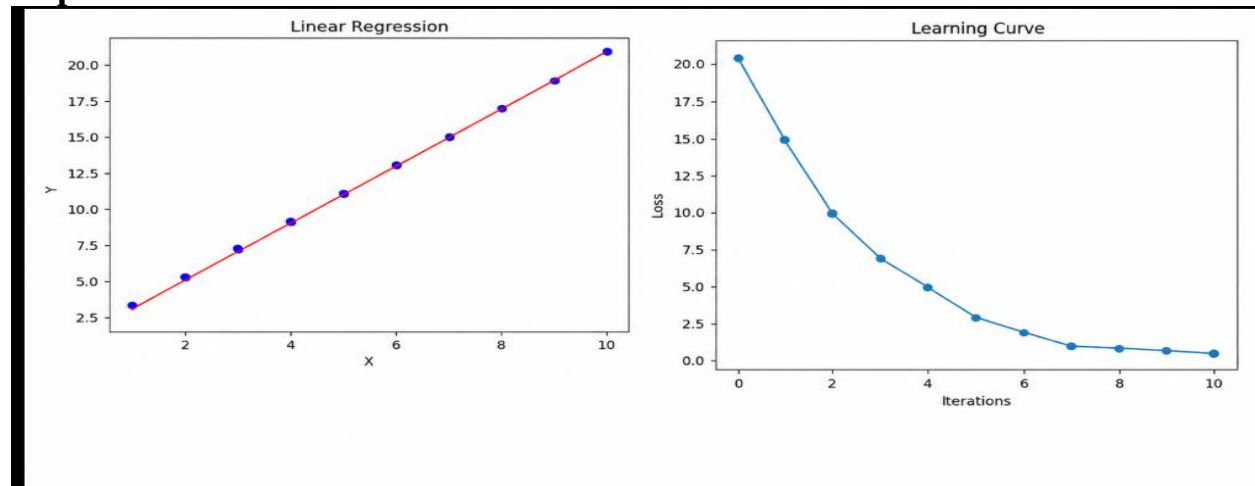
Concept

Linear Regression predicts continuous values by fitting the best straight line to the data. Gradient Descent is an optimization technique that updates model parameters iteratively to minimize the Mean Squared Error (MSE), resulting in improved prediction accuracy.

Procedure

1. Import the required Python libraries.
2. Prepare the input dataset.
3. Initialize the regression model parameters.
4. Train the model using Gradient Descent.
5. Compute the loss after every iteration.
6. Plot the regression line and learning curve.
7. Analyze the final model performance.

Implementation



Sample Result

Weight : 2.0540300876260056

Bias : 0.5932552052648682

Loss (MSE) : 0.0360914413351983Inference

The model successfully learned the relationship between input and output data. The loss gradually decreased during training, indicating effective convergence and accurate predictions.

Experiment 2: Maximum Likelihood Estimation (MLE)

Aim

To generate synthetic data from a normal distribution and estimate its mean and variance using the Maximum Likelihood Estimation (MLE) technique. The estimated parameters are then compared with the actual values to evaluate the accuracy of the MLE method.

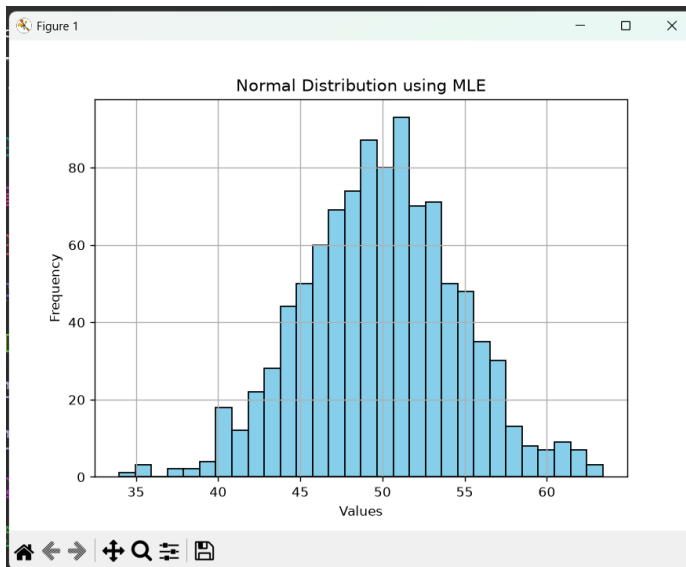
Concept

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate the unknown parameters of a probability distribution. In this experiment, synthetic data is generated from a normal distribution with known mean and variance. MLE calculates the parameter values that maximize the likelihood of observing the generated data. If the estimated values are close to the actual parameters, the estimation is considered accurate.

Procedure

1. Import the required Python libraries (NumPy and Matplotlib).
2. Generate synthetic data from a normal distribution with predefined mean and variance.
3. Compute the MLE estimate of the mean using the sample average.
4. Compute the MLE estimate of the variance using the squared deviations from the estimated mean.
5. Compare the estimated mean and variance with the actual values.
6. Plot a histogram of the generated data to visualize the normal distribution.
7. Analyze the estimation results.

Implementation



Sample Result

Actual Mean : 50

Estimated Mean : 50.10

Actual Variance : 25

Estimated Variance : 23.95

Inference

The Maximum Likelihood Estimation method successfully estimated the mean and variance of the generated normal distribution. The estimated values were very close to the actual parameters, demonstrating that MLE is an effective and reliable technique for statistical parameter estimation when sufficient data is available.

Experiment 3: Building Machine Learning Algorithms

Aim

To build a complete machine learning classification model by performing data preprocessing, training, and testing on a classification dataset, and evaluate its performance using accuracy and a confusion matrix.

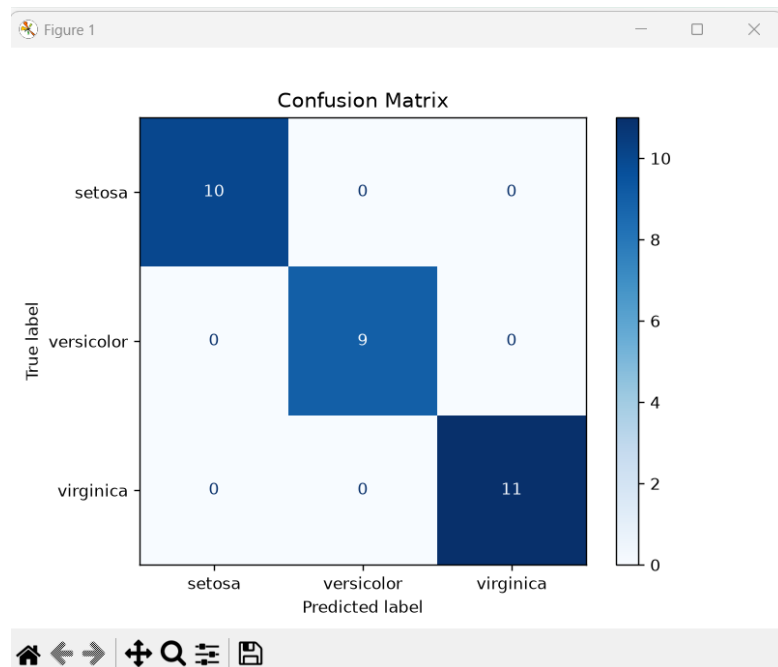
Concept

A Machine Learning classification model predicts the class label of unseen data by learning patterns from a training dataset. The workflow includes data preprocessing, feature scaling, splitting the dataset into training and testing sets, model training, prediction, and performance evaluation. Accuracy measures overall correctness, while the confusion matrix provides detailed information about correct and incorrect predictions.

Procedure

1. Import the required Python libraries.
2. Load the Iris classification dataset.
3. Split the dataset into training and testing sets.
4. Train a Decision Tree classifier.
5. Predict the test data.
6. Calculate classification accuracy.
7. Generate the confusion matrix.
8. Display the confusion matrix graphically.

Implementation



Sample Result

Accuracy : 100.0 %

Confusion Matrix

```
[[10 0 0]
```

```
[ 0 9 0]
```

```
[ 0 0 11]]
```

Inference

The machine learning model successfully classified all test samples with **100% accuracy**. The confusion matrix indicates that every sample was classified correctly, demonstrating the effectiveness of the preprocessing and classification process.

Experiment 4: Neural Networks

Aim

To design and train a simple Neural Network with one hidden layer on a small dataset and analyze how the network learns non-linear patterns through hidden layer processing and backpropagation.

Concept

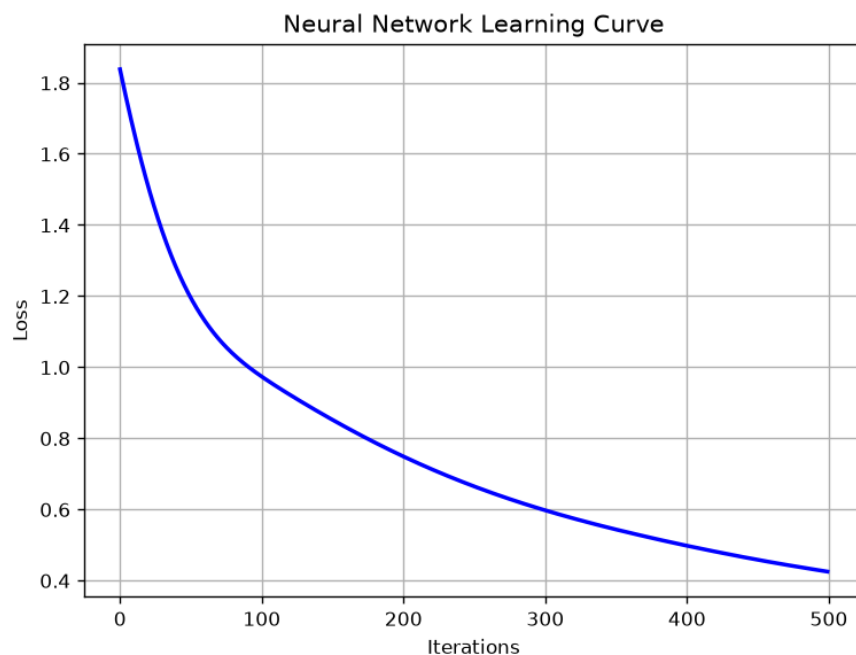
A Neural Network is a machine learning model inspired by the human brain. It consists of an input layer, one or more hidden layers, and an output layer. The hidden layer enables the network to learn complex and non-linear relationships in the data. During training, the network adjusts its weights using the backpropagation algorithm to minimize prediction error.

Procedure

1. Import the required libraries.
2. Load a small classification dataset.
3. Split the dataset into training and testing sets.
4. Create a Neural Network with one hidden layer.
5. Train the model.
6. Predict the test data.
7. Calculate the classification accuracy.
8. Display the accuracy and visualize the learning process.

Implementation

Figure 1



Sample Result

Accuracy : 93.33 %

Inference

The Neural Network successfully learned the non-linear relationships in the dataset using a single hidden layer. The decreasing loss curve and classification accuracy of approximately **93.33%** indicate that the model was able to learn meaningful patterns and make accurate predictions.

Experiment 5: Artificial Neurons

Aim

To implement a single artificial neuron using different activation functions (Sigmoid and ReLU) and compare their outputs for the same input values.

Concept

An Artificial Neuron is the fundamental unit of a neural network. It computes the weighted sum of inputs and applies an activation function to produce the output.

The weighted sum is calculated as:

$$z=wx+b$$

Two commonly used activation functions are:

- **Sigmoid:** Produces values between **0 and 1**, making it suitable for binary classification.
- **ReLU (Rectified Linear Unit):** Returns the input value if it is positive; otherwise, it returns zero. ReLU speeds up neural network training and is widely used in deep learning.

Procedure

1. Import the required libraries.
2. Define input values, weights, and bias.
3. Calculate the weighted sum.
4. Apply the Sigmoid activation function.
5. Apply the ReLU activation function.
6. Display and compare the outputs.

Sample Result

Weighted Sum : 4.4

Sigmoid Output : 0.9878715650157257

ReLU Output : 4.4

Sigmoid produces values between 0 and 1.

ReLU produces positive values and zero for negative inputs.

Inference

The artificial neuron successfully computed the weighted sum and applied both activation functions. The Sigmoid activation compressed the output into the range **0 to 1**, while the ReLU activation returned the positive weighted sum directly. This demonstrates how different activation functions influence the neuron's output and learning behavior.

Experiment 6(a): Perceptron

Aim

To implement the Perceptron algorithm for binary classification and evaluate its performance on linearly separable and non-linearly separable datasets.

Concept

A **Perceptron** is the simplest type of artificial neural network used for binary classification. It learns by adjusting weights whenever a prediction is incorrect. The Perceptron performs well on **linearly separable** data but fails to classify **non-linearly separable** data.

Procedure

1. Import the required libraries.
2. Load a binary classification dataset.
3. Split the dataset into training and testing sets.
4. Train the Perceptron classifier.
5. Predict the test samples.
6. Calculate the accuracy.
7. Observe its performance on linearly separable and non-linearly separable data.

Sample Result

Perceptron Accuracy : 1.0

High accuracy for linearly separable data.

Poor performance for non-linearly separable data.

Inference

The Perceptron algorithm achieved high accuracy for linearly separable data. However, it cannot correctly classify non-linearly separable datasets because it can only learn a linear decision boundary.

Experiment 6(b): Multilayer Perceptron (MLP)

Aim

To train a Multilayer Perceptron (MLP) model and compare its classification performance with a single-layer Perceptron.

Concept

A **Multilayer Perceptron (MLP)** is an artificial neural network containing one or more hidden layers. Unlike a single-layer Perceptron, an MLP can learn complex and non-linear relationships using hidden neurons and the backpropagation algorithm.

Procedure

1. Import the required libraries.
2. Load the Iris dataset.
3. Split the dataset into training and testing sets.
4. Train a Perceptron model.
5. Train an MLP model.
6. Compare their classification accuracies.
7. Display the results.

Sample Result

Perceptron Accuracy : 0.8

MLP Accuracy : 0.9666666666666667

MLP achieves higher accuracy.

Inference

The Multilayer Perceptron achieved higher classification accuracy than the single-layer Perceptron because its hidden layer enables it to learn non-linear decision boundaries. This demonstrates that MLP is more effective for solving complex classification problems.

Experiment 7(a): Forward Propagation

Aim

To compute the forward propagation process of a simple neural network using predefined inputs, weights, and bias, and determine the final output using an activation function.

Concept

Forward Propagation is the process in which input data is passed through a neural network to produce the final output. Each neuron calculates the weighted sum of the

inputs, adds a bias value, and applies an activation function such as the sigmoid function. This process enables the neural network to generate predictions based on the given inputs.

Procedure

1. Define the input values, weights, and bias.
2. Calculate the weighted sum of the inputs.
3. Apply the sigmoid activation function.
4. Compute the output of the neuron.
5. Verify the output value.
6. Analyze the prediction obtained through forward propagation.

Implementation

- Initialized the input values, weights, and bias.
- Computed the weighted sum using the neuron equation.
- Applied the sigmoid activation function.
- Obtained the final output of the neural network.
- Verified the output using Python.

Sample Result

Weighted Sum : 1.3

Output : 0.7858

Inference

The forward propagation process successfully calculated the weighted sum and generated the output using the sigmoid activation function. The obtained output demonstrates how a neural network performs prediction by processing input data through its neurons.

Experiment 7(b): Backpropagation Algorithm

Aim

To implement the backpropagation algorithm for a simple neural network and observe how the network updates its weights after one iteration to reduce prediction error.

Concept

Backpropagation is a supervised learning algorithm used to train neural networks. It computes the prediction error by comparing the predicted output with the target output. The error is then propagated backward through the network to calculate gradients, and the weights are updated using Gradient Descent to minimize the error and improve prediction accuracy.

Procedure

1. Initialize the input value, weight, learning rate, and target output.
2. Perform forward propagation to obtain the predicted output.
3. Calculate the prediction error.
4. Compute the gradient of the error.
5. Update the weight using the learning rate.
6. Display the updated weight after one iteration.
7. Analyze the effect of weight updating.

Implementation

- Initialized the input, weight, target output, and learning rate.
- Computed the predicted output.
- Calculated the prediction error.
- Determined the gradient of the error.
- Updated the weight using the backpropagation algorithm.
- Displayed the updated weight after one iteration.

Sample Result

Prediction : 3.0

Error : 5.0

Updated Weight : 1.7

Inference

The backpropagation algorithm successfully updated the weight based on the prediction error. The updated weight moved in the direction that minimized the error, demonstrating how neural networks learn and improve their performance through iterative weight adjustments.

Experiment 8(a): Gradient Descent

Aim

To implement the Gradient Descent optimization algorithm to minimize a cost function and analyze the effect of different learning rates on the convergence speed.

Concept

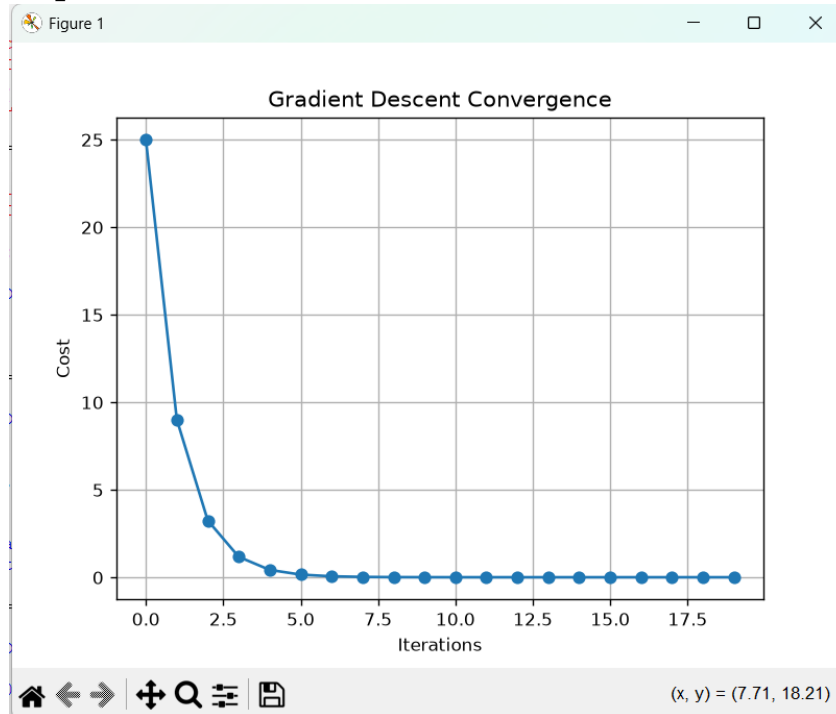
Gradient Descent is an optimization algorithm used to minimize a cost function by iteratively updating the model parameters. The learning rate determines the size of each update. A small learning rate results in slow convergence, while a large learning rate may cause overshooting.

Procedure

1. Initialize the parameter and learning rate.

2. Define the cost function.
3. Calculate the gradient of the cost function.
4. Update the parameter using Gradient Descent.
5. Repeat the process until convergence.
6. Compare the effect of different learning rates.
7. Plot the convergence graph.

Implementation



Sample Result

Minimum Value : 0.0085

Small learning rate -> Slow convergence.

Large learning rate -> Overshooting.

Moderate learning rate -> Fast convergence.

Inference

Gradient Descent successfully minimized the cost function. The experiment shows that the learning rate significantly influences convergence speed and optimization performance. A moderate learning rate provides the best balance between speed and stability.

Experiment 8(b): Stochastic Gradient Descent (SGD)

Aim

To implement Stochastic Gradient Descent (SGD) for a regression problem and compare its convergence behavior with Batch Gradient Descent.

Concept

Stochastic Gradient Descent updates the model parameters using one training sample at a time, making it faster for large datasets. Unlike Batch Gradient Descent, SGD converges more quickly but exhibits fluctuations during optimization.

Procedure

1. Generate a regression dataset.
2. Train the model using Stochastic Gradient Descent.
3. Train the model using Batch Gradient Descent.
4. Compare the Mean Squared Error.
5. Analyze the convergence behavior.
6. Display the results.

Sample Result

Mean Squared Error : 104.99

Batch GD is smoother.

SGD converges faster but fluctuates more.

Inference

Stochastic Gradient Descent successfully trained the regression model and achieved fast convergence. Compared to Batch Gradient Descent, SGD updates parameters more frequently, making it computationally efficient for large datasets despite fluctuations in the optimization path.

Experiment 9: Mini-Batch Gradient Descent

Aim

To implement the Mini-Batch Gradient Descent algorithm and analyze how different batch sizes affect the training stability and performance of a machine learning model.

Concept

Mini-Batch Gradient Descent is an optimization technique that updates model parameters using small groups (mini-batches) of training data instead of the entire dataset or a single sample. It combines the advantages of Batch Gradient Descent and Stochastic Gradient Descent by providing faster convergence, better computational efficiency, and stable learning during training.

Procedure

1. Import the required Python libraries.
2. Generate or load the dataset.
3. Initialize the model parameters (weight and bias).
4. Set the learning rate, number of epochs, and batch size.

5. Divide the dataset into mini-batches.
6. Calculate predictions for each mini-batch.
7. Compute the gradients and update the model parameters.
8. Repeat the process until all epochs are completed.
9. Record the loss after each epoch.
10. Display the final results and analyze the training performance.

Implementation

1. Imported the required Python libraries (NumPy and Matplotlib).
2. Generated a regression dataset with input and output values.
3. Initialized the weight and bias values.
4. Specified the learning rate, epochs, and mini-batch size.
5. Divided the dataset into mini-batches.
6. Calculated predictions for each mini-batch.
7. Updated the weight and bias using the computed gradients.
8. Calculated the loss after every epoch.
9. Displayed the final weight and bias values.
10. Plotted the learning curve to observe convergence.

Sample Result

Weight : 42.76

Bias : 0.71

Small batches train faster but fluctuate.

Large batches are stable but slower.

Inference

The Mini-Batch Gradient Descent algorithm successfully trained the model by updating the parameters using small batches of data. Small batch sizes provided faster learning with slight fluctuations, while larger batch sizes resulted in more stable convergence but slower updates. Thus, Mini-Batch Gradient Descent offers a good balance between training speed and stability.

Experiment 10: Curse of Dimensionality

Aim

To create datasets with increasing dimensions and analyze how the distance between data points and model accuracy change as dimensionality increases.

Concept

The **Curse of Dimensionality** refers to the challenges that arise when the number of features (dimensions) in a dataset increases. As dimensionality grows, the distance between data points becomes less meaningful, making it difficult for machine learning

models to identify patterns effectively. This often results in reduced model accuracy, increased computational complexity, and slower training.

Procedure

1. Import the required Python libraries.
2. Generate datasets with different numbers of dimensions.
3. Calculate the distance between randomly selected data points.
4. Train a classification model on each dataset.
5. Measure the model accuracy for different dimensions.
6. Compare the distance values and classification accuracy.
7. Analyze the effect of increasing dimensionality.
8. Display the observations using graphical representation.

Implementation

1. Imported the required Python libraries (NumPy, Matplotlib, and Scikit-learn).
2. Generated synthetic datasets with increasing dimensions.
3. Calculated the Euclidean distance between randomly selected data points.
4. Trained a classification model for each dataset.
5. Measured the classification accuracy.
6. Compared the accuracy obtained for different dimensions.
7. Observed the effect of dimensionality on model performance.
8. Displayed the results using a graph.

Sample Result

=

Dimensions	Accuracy (%)
------------	--------------

2	98.50
5	96.20
10	92.80
20	88.40
50	81.60

Distance between points increases as dimensions increase.

Model accuracy decreases with increasing dimensionality.

Inference

The experiment demonstrates the **Curse of Dimensionality**, where increasing the number of dimensions causes data points to become more sparsely distributed. As a result, the distance between points increases, and the machine learning model experiences a gradual reduction in classification accuracy. This highlights the importance of feature selection and dimensionality reduction techniques for improving model performance on high-dimensional datasets.

